Student Marks Management API

Implementation and verification report

Purpose: provide a runnable implementation covering the assignment requirements. This document describes the solution and the checks executed.

Scope

The solution is a small REST API using ASP.NET Core minimal APIs, Entity Framework Core, and PostgreSQL. It covers student and course management, enrollments, marks, role-based access, reporting, migrations, seed data, an HTTP smoke-test mode, and an importable Postman collection.

Reference stack

Runtime: .NET 8 / ASP.NET Core 8; **database:** PostgreSQL 16; **data access:** EF Core 8 with Npgsql; **authentication:** JWT bearer tokens and ASP.NET PasswordHasher; **API documentation:** Swagger UI.

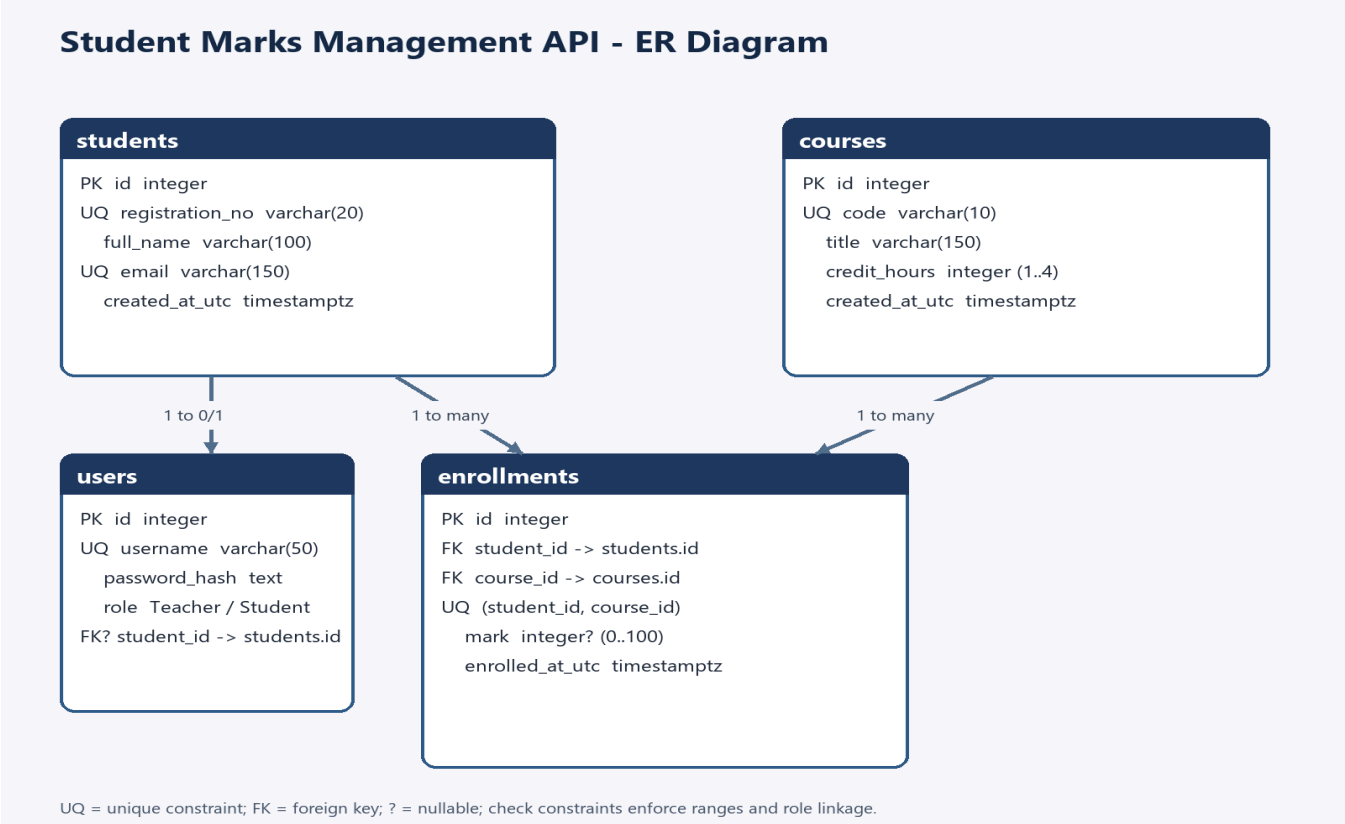
Verification snapshot

Build	PASS - 0 warnings, 0 errors
EF migration	PASS - initial schema applied to disposable PostgreSQL 16
Smoke module	PASS - 44 checks on each of two runs; zero failures
Postman collection	PASS - Newman: 43 requests, 116 assertions, zero failures
Seed and cleanup	PASS - 2 students, 3 courses, 6 enrollments, 3 users remain

Test details and reproducible commands are in docs/VERIFICATION.md.

1. Architecture and data model

The HTTP layer maps routes and authorization requirements only. Services own business rules and EF queries, are registered through interfaces, and receive cancellation tokens. DTOs are used at request/response boundaries. Read-only queries use AsNoTracking and projections. A global exception handler and status-code middleware return Problem Details rather than stack traces.



The enrollment table has a unique (student_id, course_id) constraint, so a student can enroll only once. Foreign keys restrict deletion when enrollments exist. A student user has an optional one-to-one link to its student record; the role/link check prevents Teacher accounts from carrying a student id.

2. Business rules and design decisions

Validation and persistence

• Request DTO annotations validate required values, lengths, email format, course credit hours (1-4), and marks (0-100). The API normalizes student emails to lower case; database check constraints enforce lower case as a last line of defense.

• Unique indexes enforce student registration number/email, course code, username, and the student-course enrollment pair. PostgreSQL uniqueness errors are translated to HTTP 409; no check-then-insert logic is used.

• Restricted foreign keys and service checks prevent deleting students or courses that have enrollments. Users linked to students also prevent a student deletion.

Reporting and authorization

• Teachers can perform management operations. Students can read courses and only their own record/transcript; student identity is taken from a signed JWT claim, not from caller-supplied data.

• Course statistics average graded marks only and return null for an ungraded course. Transcript weighted average is sum(mark x credit hours) divided by graded credit hours.

• Pass mark is configuration-backed (default 50). All timestamps are UTC. JWT key, database password, and demo passwords are configuration secrets and are not committed.

Error and transport behavior

• Validation errors return 400 with a camelCase errors object. Authentication/authorization, not-found, conflict, and unexpected errors use Problem Details. Unexpected errors return a generic message, while server logs retain diagnostic context.

• Successful creates return 201 and a Location header. Health returns 200 when PostgreSQL is reachable and 503 otherwise. CORS is limited to configured origins; HTTPS redirection is enabled.

3. Verification and review notes

Executed verification

- Built the project using .NET SDK 8.0.428: build succeeded with zero warnings and zero errors.
- Applied the EF Core initial migration to a disposable PostgreSQL 16 database; the service seeded the teacher, two students, three courses, and six enrollments.
- Ran the smoke mode twice against the same database. Each run reported 44 PASS checks and zero FAIL checks; test records were removed after each run.
- Checked the database after runs: exactly 2 students, 3 courses, 6 enrollments, and 3 user rows remained.

Run it locally

Follow the local setup and user-secrets instructions in README.md. From src/MarksApi, run `dotnet build`, `dotnet ef database update`, then set Development and run `dotnet run -- --smoke-test`. Start the API normally to review Swagger or run Postman.